

Processes & Services

Lecture 4

oliverni

(slide credits kmo, rrchan, coopers, keur, cs162)

Administrivia

- Vitamin 3 and Lab 3 were due last week
  - Fill out the Late Lab Form if needed.
- Lab 4 due this week
- Use your resources!
  - Your facilitators!
  - Ed, Gradescope
  - OCF Slack (ocf.io/slack) or Discord (ocf.io/discord)
  - google.com

Topics

- Processes
- Services
- Init systems
- systemd

Disclaimer

- You will explore how to use systemd in the lab!
- In lecture, I want to give more background on how processes work & why init systems exist



Goals of an operating system

- Referee
  - Manages access to shared resources (memory, cpu, etc.)
  - Isolates & protects workloads from each other



Goals of an operating system

- Referee
  - Manages access to shared resources (memory, cpu, etc.)
  - Isolates & protects workloads from each other
- Illusionist
  - Provide abstractions of physical resources
  - Each program thinks it has a dedicated machine

Goals of an operating system

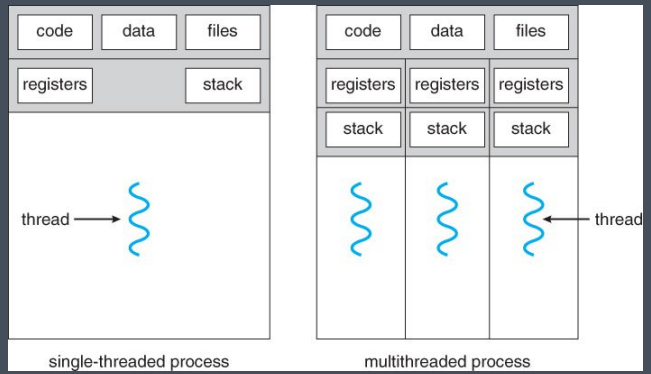
- Referee
  - Manages access to shared resources (memory, cpu, etc.)
  - Isolates & protects workloads from each other
- Illusionist
  - Provide abstractions of physical resources
  - Each program thinks it has a dedicated machine
- Glue
  - Provide common interfaces
  - File system, networking, window management, etc.

How? Process abstraction

- User code must run in a *process*
  - Referee → Restricted execution environment (privilege levels, only allowed to access what it owns)
  - Illusionist → Own virtual address space (~infinite memory from point of view of the program)
  - Glue → I/O (APIs for files, sockets, etc.)
- Each running program has its own process

Multithreaded processes

- A process may have one or more threads.
- Threads encapsulate *concurrency*
  - Share the same address space, resources, etc.
- Process are for *isolation*
  - Communication between processes much harder than between threads

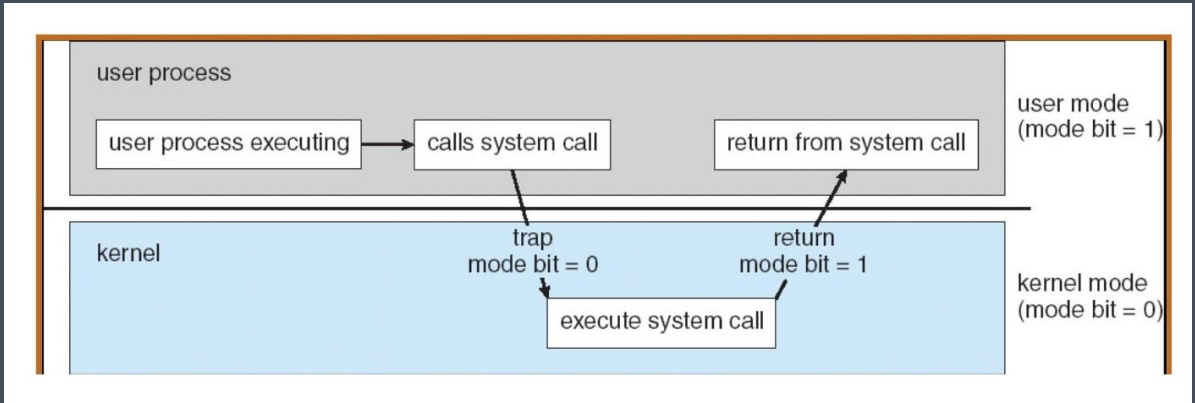


TL;DR

- A process is a *single instance of a program*
- Processes are isolated from one another and have their own memory, threads, etc.
- The OS provides common interfaces for processes to use for I/O, etc.

What about kernel code?

- Kernel code does not run in a process



What creates processes?



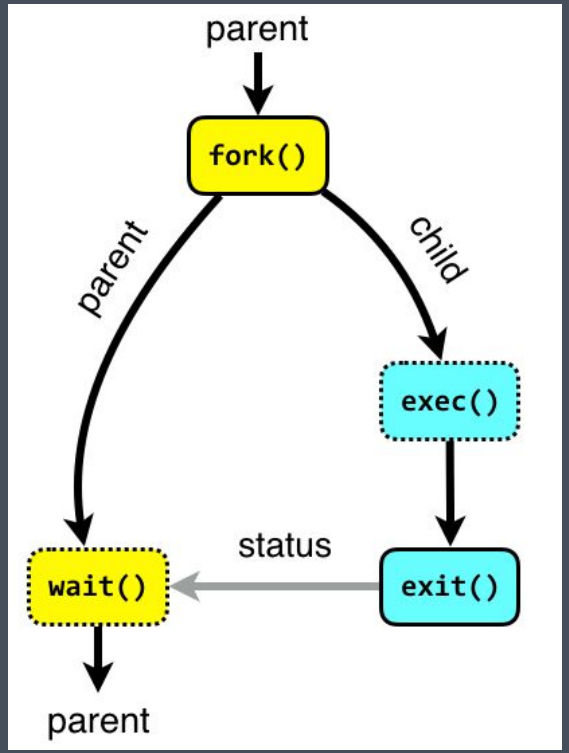
What creates processes?

Processes!

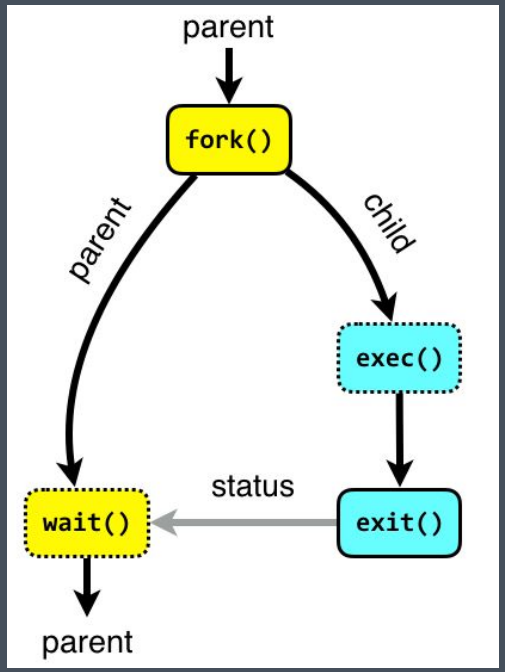


What creates processes?

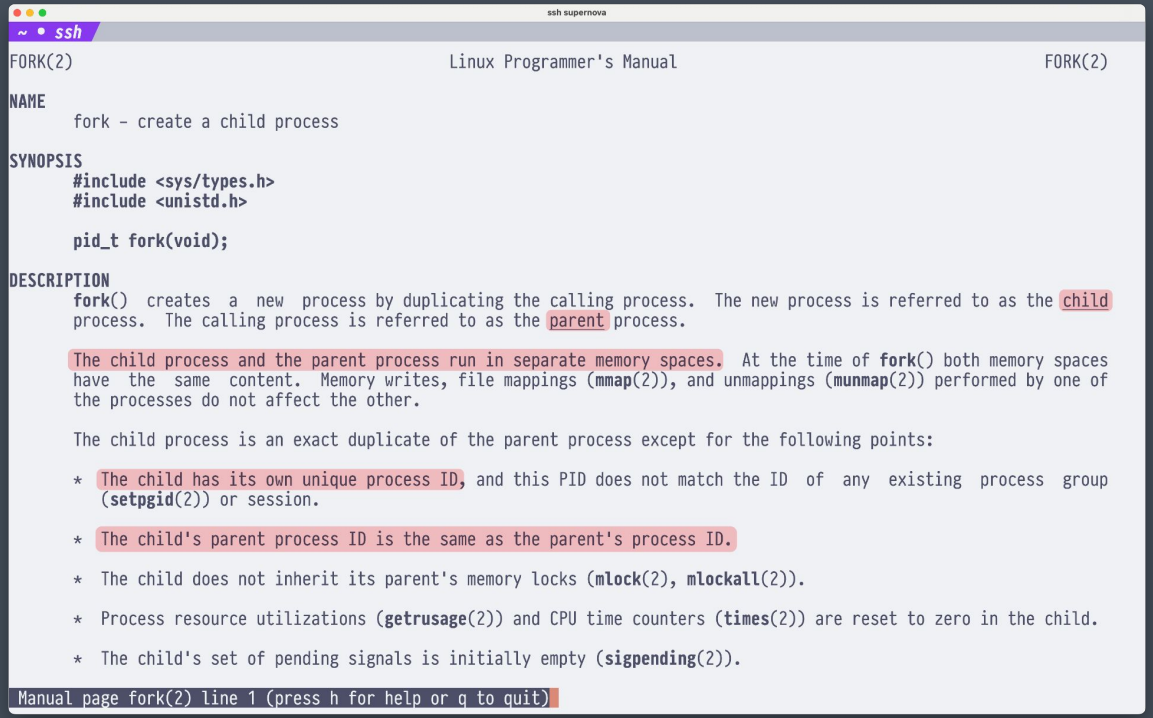
Processes!



fork-join pattern



- ***fork*:** creates a copy of the current process
- ***exec*:** restart current process with a different executable
- ***wait*:** wait for a process to finish, retrieves the exit status


ssh supernova
~ • ssh
FORK(2) Linux Programmer's Manual FORK(2)
NAME
fork - create a child process
SYNOPSIS
#include <sys/types.h>
#include <unistd.h>
pid_t fork(void);
DESCRIPTION
fork() creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.
The child process and the parent process run in separate memory spaces. At the time of fork() both memory spaces have the same content. Memory writes, file mappings (mmap(2)), and unmappings (munmap(2)) performed by one of the processes do not affect the other.
The child process is an exact duplicate of the parent process except for the following points:
* The child has its own unique process ID, and this PID does not match the ID of any existing process group (setpgid(2)) or session.
* The child's parent process ID is the same as the parent's process ID.
* The child does not inherit its parent's memory locks (mlock(2), mlockall(2)).
* Process resource utilizations (getrusage(2)) and CPU time counters (times(2)) are reset to zero in the child.
* The child's set of pending signals is initially empty (sigpending(2)).
Manual page fork(2) line 1 (press h for help or q to quit)

man 2 fork

Processes form a *tree*

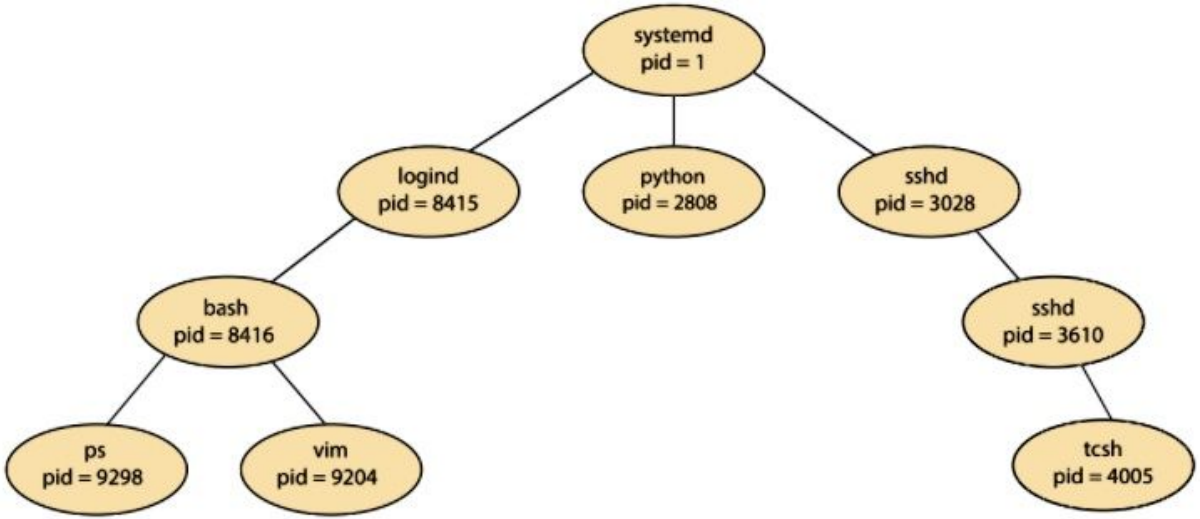


Figure 3.7 A tree of processes on a typical Linux system.

How many “henlo”s get printed?

```
int main( void ) {
    fork();
    printf( "henlo: %d\n", getpid() );
}
```



How many “henlo”s get printed?

PID 1000

```
int main( void ) {
    fork();
    printf( "henlo: %d\n", getpid() );
}
```

PID 1000

```
int main( void ) {
    fork();
    printf( "henlo: %d\n", getpid() );
}
```

PID 2000

```
int main( void ) {
    fork();
    printf( "henlo: %d\n", getpid() );
}
```



How many “henlo”s get printed?

```
$ ./fork
henlo 104632
henlo 104635
henlo 104634
henlo 104633
henlo 104638
henlo 104639
henlo 104637
henlo 104636
```

```
int main( void ) {
    fork();
    fork();
    fork();
    printf( "henlo: %d\n", getpid() );
}
```

2^1

2^2

2^3

Have you seen this code before?

```
:(){ :|:& };:
```

```
bomb() {
    bomb | bomb &
} bomb
```

Making the child do something

```
int main( void ) {
    if ( fork() > 0 ) {
        /* parent process */
        wait( NULL );
    } else {
        /* child process */
        execv( “/bin/bash”, NULL );
    }
}
```

```
/usr/bin/zsh
 \_ ./fork_exec
     \_ [bash]
```

htop exploration

- htop(1): interactive terminal process manager
- https://peteris.rocks/blog/htop/

- PID/User/Command/CPU%/MEM%/TIME: self-explanatory
- PRI/NI: Priority and Niceness (take cs162, or google scheduling policy/SCHED_OTHER)
- VIRT/RES/SHR: virtual image, resident size, shared memory usage
- S: Status (S = sleep, R = running, T = terminated, Z = zombie)

Chicken and egg problem?

- If processes create other processes, what is the first process, and how is it created?



Chicken and egg problem?

- If processes create other processes, what is the first process, and how is it created?
  - The kernel creates the *init* process during boot



Init process

- Started by kernel during boot
- Manages all other services and processes

What happens when a process dies?

```
if ( fork() > 0 ) {
    /* parent process */
    sleep( 1 );
} else {
    /* child process */
    exit( 1 );
}
```

When a process exits, returns int (exit code 0-255)

0 is success and anything else indicates an error

Parents need to wait() on children

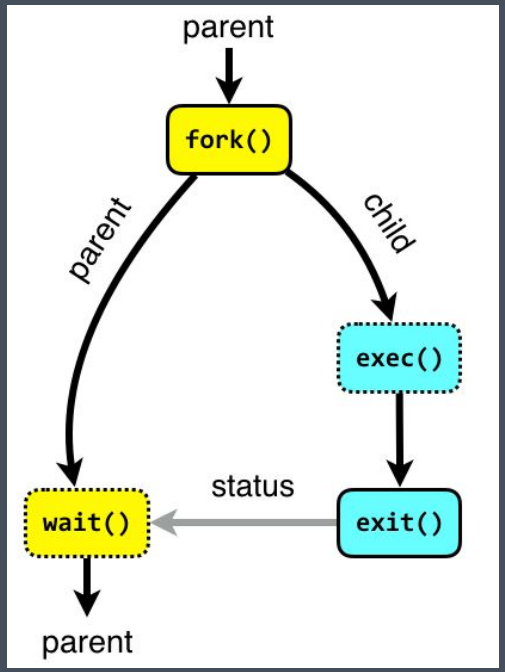
```c
int main( void ) {
    if ( fork() > 0 ) {
        int status;
        sleep( 1 );
        wait( &status );
        printf( "%d\n", WEXITSTATUS ( status ) );
    } else {
        exit( 123 );
    }
```

```
# keur @ magneto in ~/repos/temp
$ ./good-parent
123
```

What happens if the parent doesn’t wait?

- OS needs to keep exit status somewhere, even if the parent doesn’t wait
- If parent never waits, this causes a resource leak!

fork-join pattern



- ***fork*:** creates a copy of the current process
- ***exec*:** restart current process with a different executable
- ***wait*:** wait for a process to finish, retrieves the exit status

What happens if the parent exits first?

```c
int main( void ) {
    if ( fork() > 0 ) {
        /* parent process */
        sleep ( 1 );
        exit( 0 );
    } else {
        /* child process */
        sleep( 100 );
        exit( 123 );
    }
```

Process tree after start

Parent

Child

```
\_ -zsh
   \_ ./forking
      \_ ./forking
```

Process tree after 1 second

Parent

Child

```
/sbin/init
 \_ ./forking
```

Init responsibilities: Manage processes

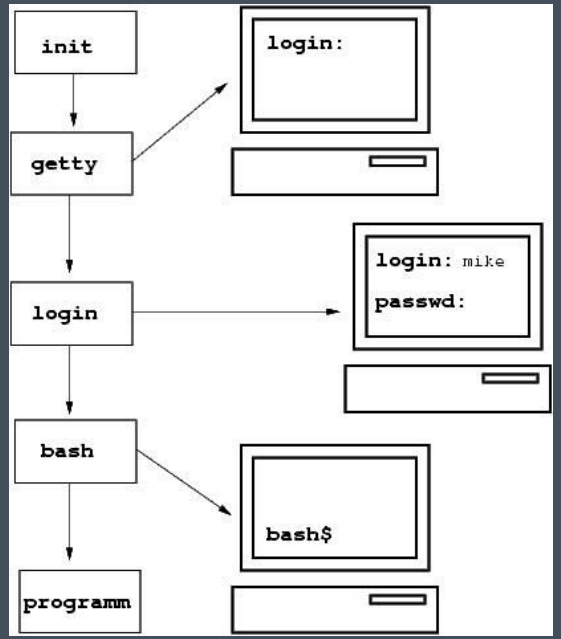
- If parent exits first, orphan processes are re-parented to the init process
- Init waits on all processes
  - Zombie processes can still occur if the parent never exits!



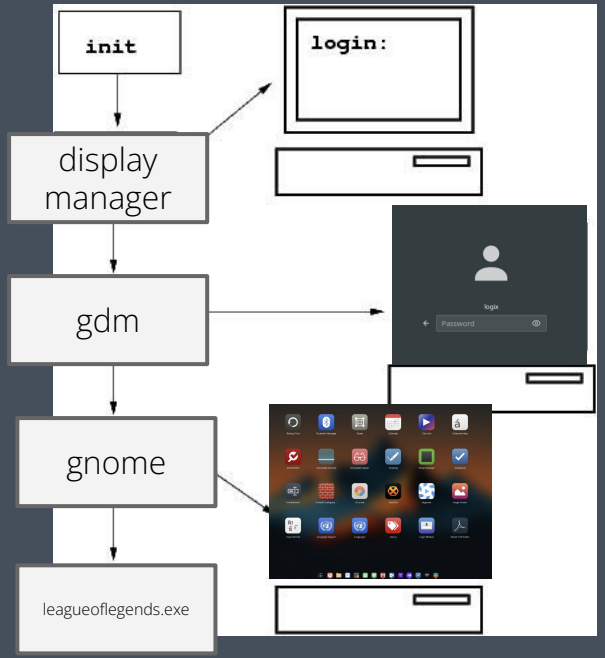
Init responsibilities: Start critical services

- We need a way to access the computer!
- Critical services include
  - getty
  - display manager
  - ssh

Text login process



Graphical login process



Init responsibilities: Manage services

- A process that needs to be kept track of
- Often *dameons*, non-interactive background processes
- getty, sddm, sshd, httpd, nginx, postfix, etc.
- Need to be always running!
  - If something is not running, we might not be able to access our computer

systemd

- An init system
  - Most commonly used init system on modern Linux systems
- Provides tools for users to manage services
  - systemctl - start, stop, check status, and more
  - journalctl - check systemd journal
- Also manages a lot more!
  - logind, networkd, resolved, tmpfilesd, homed, ...

systemd unit files

- Service behavior defined by systemd unit files
  - How should this service start up? How should it respond to various systemctl management commands?

A unit file

```
[Unit]
Description=A simple unit file

[Service]
ExecStart=/usr/bin/helloworld
User=ocfstaff
Restart=always

[Install]
WantedBy=multi-user.target
```

Description - what the service does

Commands

When this unit should get started



Another unit file

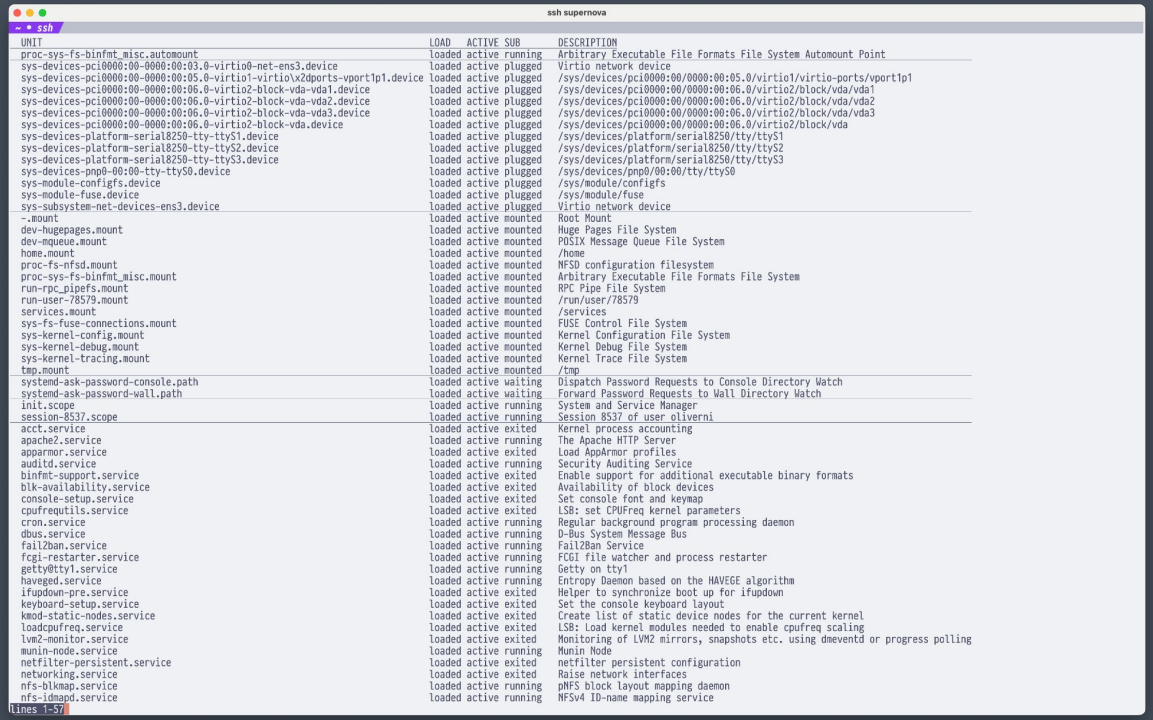
```
oliverni@death:~$ systemctl cat apache2
# /lib/systemd/system/apache2.service
[Unit]
Description=The Apache HTTP Server
After=network.target remote-fs.target nss-lookup.target
Documentation=https://httpd.apache.org/docs/2.4/

[Service]
Type=forking
Environment=APACHE_STARTED_BY_SYSTEMD=true
ExecStart=/usr/sbin/apachectl start
ExecStop=/usr/sbin/apachectl graceful-stop
ExecReload=/usr/sbin/apachectl graceful
KillMode=mixed
PrivateTmp=true
Restart=on-abort

[Install]
WantedBy=multi-user.target
```



Lots of units!



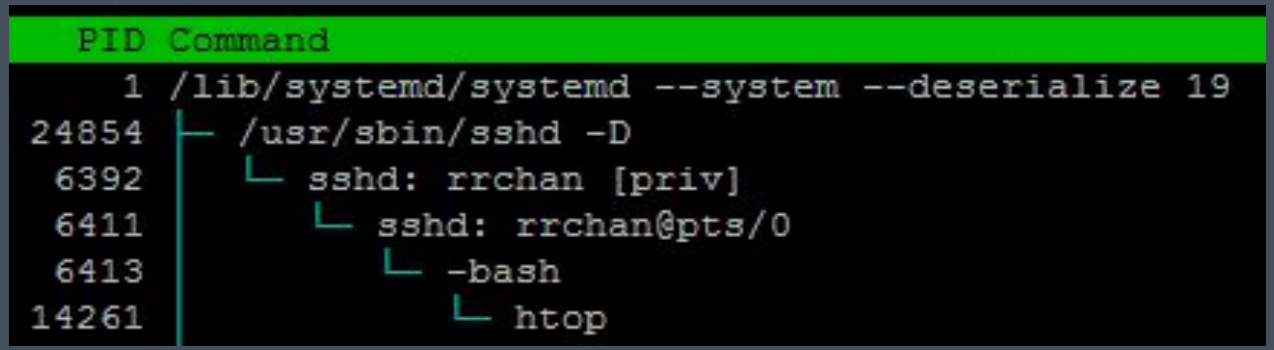
systemd commands

- `systemctl start [name]` - starts a service
- `systemctl stop [name]` - stops a service
- `systemctl restart [name]` - restarts a service
- `systemctl reload [name]` - reload a service's configuration
- `systemctl enable [name]` - sets a service to start on boot
- `systemctl disable [name]` - opposite of *enable*

Behavior for these commands may depend on the service, as defined in the unit file!

Example - sshd

- Handles ssh connections



- Notice that *bash* and *htop* are children of sshd, since rrchan is ssh'd into this machine.